

✓ Practical 02: Your First Honest Model

Split the data, establish a baseline, train a model, and evaluate it on held-out observations

SCSE3040 Machine Learning Operations | Bennett University | Session 2026-27

Item	Detail
Lecture	L03
Course Outcome	CO2
Duration	120 minutes
Memory	about 300 MB
GPU	not required
Marks	10

Aim

1. Separate features from the prediction target.
2. Establish a reproducible train/test evaluation boundary.
3. Build a training-only baseline.
4. Fit a Linear Regression model.
5. Evaluate held-out predictions using MAE and RMSE.
6. Compare model evidence with the baseline.
7. Build a small reusable inference function.

The lesson is not simply Linear Regression. The lesson is that a model score needs an evaluation protocol and a reference point before it becomes meaningful evidence.

✓ Before you start

P01 should already be complete. You should understand the course environment, the shared dataset, and why random seeds matter.

Run cells **top to bottom** with **Shift + Enter**.

If notebook state becomes confusing, use **Kernel > Restart Kernel and Clear All Outputs**, then begin again.

A later cell may depend on objects created earlier. A `NameError` often means a prerequisite cell was skipped.

```
# Step 0: verify the execution context before modelling.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Program :", sys.executable)
print("Folder  :", Path.cwd())

_missing = []
for _name in ["numpy", "pandas", "sklearn"]:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ["numpy", "pandas", "sklearn"]:
    print(f"  {_name:<14} {'missing' if _name in _missing else 'ok'}")

if _missing:
    print()
    print("STOP. Missing imports:", ", ".join(_missing))
    print("Verify that Jupyter is using the intended course Python.")
else:
    print()
    print("All good. Continue to Step 1.")
```

```
Python   : 3.11.9
Program  : /home/manvi/Documents/academics/ML_Ops/Lab1/.venv/bin/python
Folder   : /home/manvi/Documents/academics/ML_Ops/Lab2_S24CSEU0266
numpy    ok
pandas   ok
sklearn  ok
```

```
All good. Continue to Step 1.
```

✓ Step 0b: ensure the shared dataset exists

Every practical uses the same 600 synthetic delivery observations.

If the shared CSV is absent, the next cell reconstructs the teaching dataset from the course seed. This is a repository convenience, not a recommendation to silently regenerate missing production data.

```
import csv
from pathlib import Path

import numpy as np
```

```
SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"

def make_delivery_csv(path=DATA, seed=SEED):
    """Write the 600-row synthetic course dataset."""
    rng = np.random.default_rng(seed)

    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)

    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )

    path.parent.mkdir(parents=True, exist_ok=True)

    with path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow([
            "distance_km",
            "prep_time_min",
            "traffic_level",
            "rain",
            "delivery_min",
        ])

        for i in range(N_ROWS):
            writer.writerow([
                distance_km[i],
                int(prep_time_min[i]),
                int(traffic_level[i]),
                int(rain[i]),
                delivery_min[i],
            ])

    return path

if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found ->", DATA)
```

```
dataset found -> ../data/delivery_times.csv
```

✓ Walkthrough

Read each step, then run its cell.

✓ Step 1: load and inspect the deliveries

Never train a model on a table whose basic structure you have not inspected.

Check its shape, first rows, column names, and data types before deciding what becomes a feature or target.

```
import numpy as np
import pandas as pd

orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())
print()
print("data types:")
print(orders.dtypes)
```

```
rows, columns: (600, 5)
```

	distance_km	prep_time_min	traffic_level	rain	delivery_min
0	9.40	17	1	0	51.3
1	5.55	24	2	1	54.2
2	10.37	28	3	0	67.7
3	8.52	23	2	0	51.2
4	1.58	29	2	1	42.1

```
data types:
distance_km      float64
prep_time_min    int64
traffic_level    int64
rain             int64
delivery_min     float64
dtype: object
```

The dataset has five columns. Four describe information available before the final delivery duration is observed. `delivery_min` is the outcome to predict.

Before continuing, make sure you can explain what one row represents.

✓ Step 2: separate features from the target

X contains the model inputs. y contains the prediction target.

The names X and y are common mathematical and machine-learning conventions, but the important idea is the separation of input information from the outcome being learned.

```
FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
TARGET = "delivery_min"
```

```
X = orders[FEATURES]
y = orders[TARGET]
```

```
print("X shape:", X.shape, "  <- 600 orders, 4 features each")
print("y shape:", y.shape, "    <- 600 answers")
print()
print(X.head(3))
print()
print(y.head(3))
```

```
X shape: (600, 4)  <- 600 orders, 4 features each
y shape: (600,)   <- 600 answers
```

	distance_km	prep_time_min	traffic_level	rain
0	9.40	17	1	0
1	5.55	24	2	1
2	10.37	28	3	0

```
0    51.3
1    54.2
2    67.7
```

```
Name: delivery_min, dtype: float64
```

✓ Step 3: create the evaluation boundary

Reserve 20% of the observations for held-out evaluation.

`random_state=42` makes this particular partition reproducible.

The test set is withheld from `.fit()`. It is not a second training set.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print("training on:", len(X_train), "orders")
print("testing on :", len(X_test), "orders")
print("total      :", len(X_train) + len(X_test))
```

```

training on: 480 orders
testing on  : 120 orders
total      : 600

```

From this point, `X_test` and `y_test` are not supplied to `model.fit()`.

This creates the first evaluation boundary:

```

training observations -> parameter learning
held-out observations -> evaluation evidence

```

Later, P03 will show why repeatedly consulting the same test set while choosing models can also compromise its independence.

✓ Step 4: establish the number to beat

Before fitting the regression model, define a simple baseline.

The baseline ignores all four features and always predicts the **mean delivery time from the training targets**.

The phrase *from the training targets* matters. Using `y_test` to construct the baseline would allow held-out target information to influence the predictor.

```

from sklearn.metrics import mean_absolute_error, mean_squared_error

average_time = y_train.mean()
print(f"average delivery time in training data: {average_time:.1f}")

baseline_guesses = np.full(len(y_test), average_time)
baseline_mae = mean_absolute_error(y_test, baseline_guesses)

print(f"BASELINE MAE: {baseline_mae:.2f} minutes")
print()
print("Meaning: guessing the average is wrong by about")
print(f"{baseline_mae:.0f} minutes on a typical order.")

```

```

average delivery time in training data: 47.1 min
BASELINE MAE: 10.32 minutes

```

```

Meaning: guessing the average is wrong by about
10 minutes on a typical order.

```

✓ Step 5: fit Linear Regression

`LinearRegression()` creates an unfitted estimator.

`.fit(X_train, y_train)` estimates its parameters using the training observations

observations.

The model remains deliberately simple because P02 is about evaluation discipline, not algorithmic sophistication.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model trained on", len(X_train), "orders.")
print("It has learned", len(model.coef_), "numbers, one per feature")
```

```
Model trained on 480 orders.
It has learned 4 numbers, one per feature.
```

✓ Step 6: evaluate held-out predictions

We report:

- **MAE**: the mean absolute prediction error, expressed in minutes.
- **RMSE**: the square root of the mean squared error, also expressed in minutes.

RMSE gives larger residuals more influence because errors are squared before averaging.

Do not conclude from $\text{RMSE} > \text{MAE}$ alone that there must be a few catastrophic predictions. For non-identical absolute residuals, RMSE is generally at least MAE. Stronger claims require inspection of the residual distribution.

```
predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = float(np.sqrt(mean_squared_error(y_test, predictions)))

print(f"BASELINE MAE : {baseline_mae:6.2f} minutes")
print(f"MODEL      MAE : {mae:6.2f} minutes")
print(f"MODEL      RMSE : {rmse:6.2f} minutes")
print()
improvement = 100 * (baseline_mae - mae) / baseline_mae
print(f"The model is {improvement:.0f}% better than guessing.")
```

```
BASELINE MAE : 10.32 minutes
MODEL      MAE : 1.92 minutes
MODEL      RMSE : 2.48 minutes
```

```
The model is 81% better than guessing.
```

A useful report is not merely:

MAE is 2.

It should include context such as the held-out evaluation design and baseline.

For example:

The model obtained an MAE of about 2 minutes on the held-out test set, compared with a training-mean baseline MAE of about 10 minutes.

That is substantially more informative evidence.

✓ Step 7: inspect the fitted coefficients

Linear Regression estimates one coefficient per feature.

Within this fitted model, a coefficient describes the change in predicted delivery time associated with a one-unit change in that feature while the other included features are held fixed.

A coefficient is a model parameter. It should **not automatically be interpreted as a causal effect**.

```
learned = pd.DataFrame({
    "feature": FEATURES,
    "minutes added per unit": model.coef_.round(2),
})

print(learned.to_string(index=False))
print()
print(f"starting point (intercept): {model.intercept_:.1f} minutes")
```

feature	minutes added per unit
distance_km	3.07
prep_time_min	0.65
traffic_level	4.13
rain	5.55

starting point (intercept): 6.4 minutes

Because this is a synthetic dataset generated from an approximately linear formula, the learned coefficients should be close to the values used by the generator.

That agreement is useful for teaching, but in real observational data a plausible-looking coefficient is not proof that the pipeline is correct or that the relationship is causal.

✓ Step 8: predict one new order

Use the model as an application eventually would: supply one new feature record and receive one predicted duration.

A one-row DataFrame preserves the same named feature interface used during training.

```
new_order = pd.DataFrame([{\n    "distance_km": 5.0,\n    "prep_time_min": 20,\n    "traffic_level": 2,\n    "rain": 0,\n}])\n\nminutes = model.predict(new_order)[0]\nprint(f"Predicted delivery time: {minutes:.1f} minutes")
```

Predicted delivery time: 43.0 minutes

Keep this inference pattern in mind:

```
caller inputs -> feature representation -> model.predict() -> returned
```

P07 will place essentially this boundary behind an HTTP endpoint.

✓ Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

✓ Task T1: compare mean and median baselines

Step 4 used the training-set mean.

Build a second baseline that predicts `y_train.median()` for every held-out order.

Store its MAE in:

```
T1_median_mae
```

Then set:

```
T1_which_is_better
```

to exactly "mean" or "median", whichever has the lower MAE.

If the difference is tiny, do not exaggerate its practical importance.

```
median_guesses = np.full(len(y_test), y_train.median())
T1_median_mae = mean_absolute_error(y_test, median_guesses)

T1_which_is_better = "mean" if baseline_mae < T1_median_mae else "r

print("median baseline MAE:", T1_median_mae)
print("better baseline      :", T1_which_is_better)

median baseline MAE: 10.331666666666669
better baseline      : mean
```

✓ Task T2: test sensitivity to another split

Repeat the complete experiment using:

```
test_size = 0.30
random_state = 7
```

Create `X_tr2, X_te2, y_tr2, y_te2`, fit a **new** `LinearRegression`, and store its held-out MAE in `T2_mae`.

A new split changes the training observations, so the intended experiment requires a new fitting step.

This is only a first sensitivity check. Two manually selected splits do not replace cross-validation.

Task T2 answer. The conclusion did not change: with the 70/30 split and seed 7, the linear model again has a much lower MAE than the simple baselines. This variation confirms that a single split is only an estimate influenced by its held-out observations, so model evaluation should be checked with multiple splits or cross-validation.

```
X_tr2, X_te2, y_tr2, y_te2 = train_test_split(
    X, y, test_size=0.30, random_state=7
)
```

```

model2 = LinearRegression().fit(X_tr2, y_tr2)
T2_mae = mean_absolute_error(y_te2, model2.predict(X_te2))

print("70/30 split, seed 7 -> MAE", T2_mae)

70/30 split, seed 7 -> MAE 2.096334872408644

```

✓ Task T3: one order in, one number out

Implement:

```
predict_minutes(distance_km, prep_time_min, traffic_level, rain)
```

It must build a one-row DataFrame with the correct feature names, call the original fitted `model`, and return one plain number rounded to one decimal place.

Then call it for:

```

distance_km    = 3
prep_time_min  = 15
traffic_level  = 1
rain           = 1

```

and store the result in `T3_rainy_order`.

This small function is the first reusable inference interface in the course.

```

def predict_minutes(distance_km, prep_time_min, traffic_level, rain):
    order = pd.DataFrame([
        "distance_km": distance_km,
        "prep_time_min": prep_time_min,
        "traffic_level": traffic_level,
        "rain": rain,
    ])
    return round(float(model.predict(order)[0]), 1)

T3_rainy_order = predict_minutes(3, 15, 1, 1)

print("rainy 3 km order ->", T3_rainy_order, "minutes")

results_table = pd.DataFrame({
    "model": ["Mean baseline", "Median baseline", "Linear regression"],
    "test MAE (minutes)": [baseline_mae, T1_median_mae, mae],
}).round(2)
print("\nSubmission results table:")
print(results_table.to_string(index=False))

rainy 3 km order -> 35.0 minutes

```

Submission results table:

	model	test MAE (minutes)
Mean baseline		10.32
Median baseline		10.33
Linear regression		1.92

✓ Self-check

Run the cell below to mark your work.

```
# -----
# SELF-CHECK
# -----

_results = []

def _check(label, fn):
    """Evaluate one condition without breaking the notebook."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))

median_mae_reference = mean_absolute_error(
    y_test,
    np.full(len(y_test), y_train.median()),
)

_check(
    "T1 | T1_median_mae is the MAE of a median baseline",
    lambda: abs(float(T1_median_mae) - median_mae_reference) < 0.01
)

_check(
    "T1 | T1_which_is_better names the lower-MAE baseline",
    lambda: T1_which_is_better
    == ("mean" if baseline_mae < median_mae_reference else "median")
)

_check(
    "T2 | the new test set holds 30% of the orders",
    lambda: len(X_te2) == 180,
)

_check(
    "T2 | model2 is a trained LinearRegression",
    lambda: isinstance(model2, LinearRegression)
```

```

        and hasattr(model2, "coef_")
        and len(model2.coef_) == 4,
    )

    _check(
        "T2 | T2_mae is model2's MAE on the new held-out set",
        lambda: abs(
            float(T2_mae)
            - mean_absolute_error(y_te2, model2.predict(X_te2))
        ) < 0.01,
    )

    _check(
        "T3 | predict_minutes returns one plain number",
        lambda: isinstance(predict_minutes(5.0, 20, 2, 0), (int, float))
    )

    reference_order = pd.DataFrame([
        {"distance_km": 5.0,
         "prep_time_min": 20,
         "traffic_level": 2,
         "rain": 0,
        }])

    _check(
        "T3 | it agrees with the trained model after one-decimal roundi",
        lambda: abs(
            predict_minutes(5.0, 20, 2, 0)
            - round(float(model.predict(reference_order)[0]), 1)
        ) < 1e-9,
    )

    _check(
        "T3 | rain increases this synthetic order's fitted prediction",
        lambda: predict_minutes(3.0, 15, 1, 1)
        > predict_minutes(3.0, 15, 1, 0),
    )

    _check(
        "T3 | T3_rainy_order is the requested rainy-order prediction",
        lambda: abs(
            float(T3_rainy_order)
            - predict_minutes(3.0, 15, 1, 1)
        ) < 1e-9,
    )

    print("=" * 68)
    print("SELF-CHECK   Practical 02: Your First Honest Model")
    print("=" * 68)

    for _label, _ok in _results:
        print(f"   [{'PASS' if _ok else 'FAIL'}]   {_label}")

    print("-" * 68)

```

```

_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("=" * 68)

if _passed == len(_results):
    print("Well done. Save the notebook using the required P02 file
else:
    print("Read the FAIL lines, fix only those tasks, then run this

```

```

=====
SELF-CHECK   Practical 02: Your First Honest Model
=====

```

```

[PASS] T1 | T1_median_mae is the MAE of a median baseline
[PASS] T1 | T1_which_is_better names the lower-MAE baseline
[PASS] T2 | the new test set holds 30% of the orders
[PASS] T2 | model2 is a trained LinearRegression
[PASS] T2 | T2_mae is model2's MAE on the new held-out set
[PASS] T3 | predict_minutes returns one plain number
[PASS] T3 | it agrees with the trained model after one-decimal ro
[PASS] T3 | rain increases this synthetic order's fitted predicti
[PASS] T3 | T3_rainy_order is the requested rainy-order predicti

```

```

-----
  9 of 9 checks passed
=====

```

```

Well done. Save the notebook using the required P02 filename.

```